

All Digital Phase Locked Loop

Submitted by

Team ID: elec_7

Name	Roll Number
Rachit Chowhan	250002055
Pranav Suresh	250002052
Yugal Tikale	250002086
Aneesh Vinayak	250002012

Contents

1 Introduction 1

2 System Architecture and Block Diagram 1

3 High Frequency ADPLL architecture 2

4 FPGA Implementation Challenges and Constraints 2

5 FPGA-Oriented ADPLL Architecture 2

6 Bonus Tasks 3

6.1 Clock Domain Crossing (CDC) via Asynchronous FIFO 3

6.2 Adaptive "Gear Shifting" Loop Filter 3

6.3 Fractional-N Synthesis via Sigma Delta Modulation 3

7 Extra features 4

7.1 Frequency Locked Loop(FLL) 4

7.2 CIC Decimator 5

7.3 Power Optimization 5

8 Results and Performance Metrics 6

8.1 System Operating Parameters 6

8.2 Transient Response and Lock Dynamics 6

8.3 Vivado Implementation 7

8.4 Jitter and Time-Domain Analysis 9

8.5 Gate Level Simulation 9

9 Limitations and Future Work 9

10 Conclusion 10

1 Introduction

An All-Digital Phase-Locked Loop (ADPLL) is a clock generation and synchronization system that replaces the analog components of a conventional Phase-Locked Loop (PLL) with digital circuitry. The primary purpose of an ADPLL is to generate a stable, high-frequency clock that remains synchronized with a lower-frequency reference clock. ADPLLs have become increasingly important in modern System-on-Chips (SoCs), communication systems, processors, and high-speed interfaces because digital circuits scale efficiently with advancing semiconductor technologies.

2 System Architecture and Block Diagram

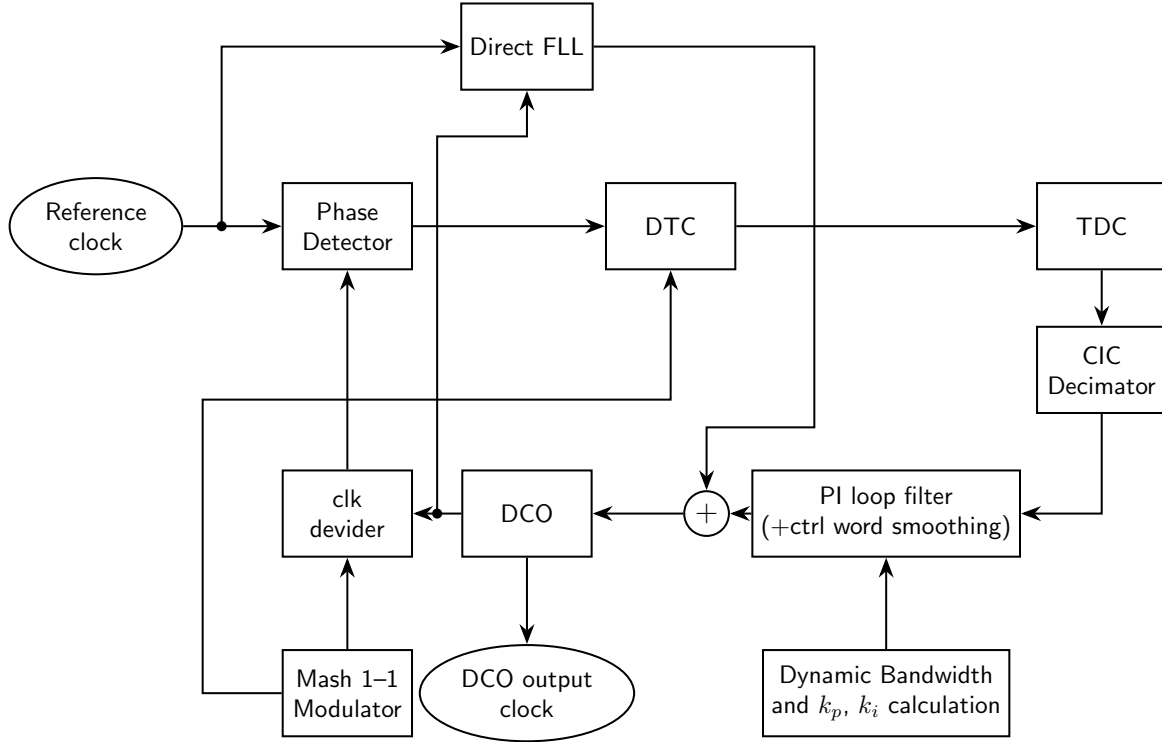


Figure 1: System Architecture Flowchart

The All-Digital Phase-Locked Loop (ADPLL) is a closed-loop system which synchronizes an output oscillator with a stable reference clock. The operation begins at the phase detector, which continuously compares the incoming reference clock against a divided version of the digitally controlled oscillator (DCO) output. The loop initially uses a FLL (frequency locked loop). The FLL counts the high-speed oscillator edges across 16 reference clock cycles. It calculates the frequency error by comparing this count against the target count. The module outputs a coarse frequency control word, this process continues until a tight threshold is reached, thus reducing capture time. For fractional inputs ($K_{mod} \neq 0$), a mash modulator is used that utilizes a sigma delta modulator to dynamically dither the clock divider value, thus achieving fractional average overtime. This dithering introduced by phase noise which is eliminated by a Digital-to-Time Converter model (dtt_model). It computes the expected quantization error and subtracts it from the combined phase error leading to a clean phase error (phase_residual).

This phase residual travels to the CIC decimator which acts like a down sampler for phase residual and a low-pass filter. A gain scheduler is used to dynamically shift the K_i , K_p values (Gear shifting) based on the phase error. It has 4 modes- (FLL, LARGE, MEDIUM, FINE) to balance rapid acquisition with low steady-state jitter. The PI-filter applies these dynamic gains to the phase error using a Proportional-Integral (PI) control scheme. An integrator adds up the error over time including slew-rate limiting to prevent sudden massive jumps in the fine tuning word. The DCO takes the control word to generate a output frequency. It also internally tracks an over sampled fine-phase value to feed back to the TDC. The clock divider divides the high-speed DCO clock down by the integer N_{div} value, outputting the feedback clock to close the loop back at the phase detector.

3 High Frequency ADPLL architecture

The architecture uses a 100MHz reference clock and uses a digitally controlled oscillator(DCO) with a free running frequency of 3.5 GHz. The DCO output frequency can be tuned over a range of 3GHz to 4GHz, giving it enough room to counter PVT variations. A python model followed by a behavioral RTL of the complete system was developed to evaluate loop dynamics, frequency acquisition, phase tracking, locking behavior, and jitter performance. This high-frequency architecture was used as a baseline design to validate the functionality and performance of the proposed ADPLL before considering practical hardware implementation constraints.

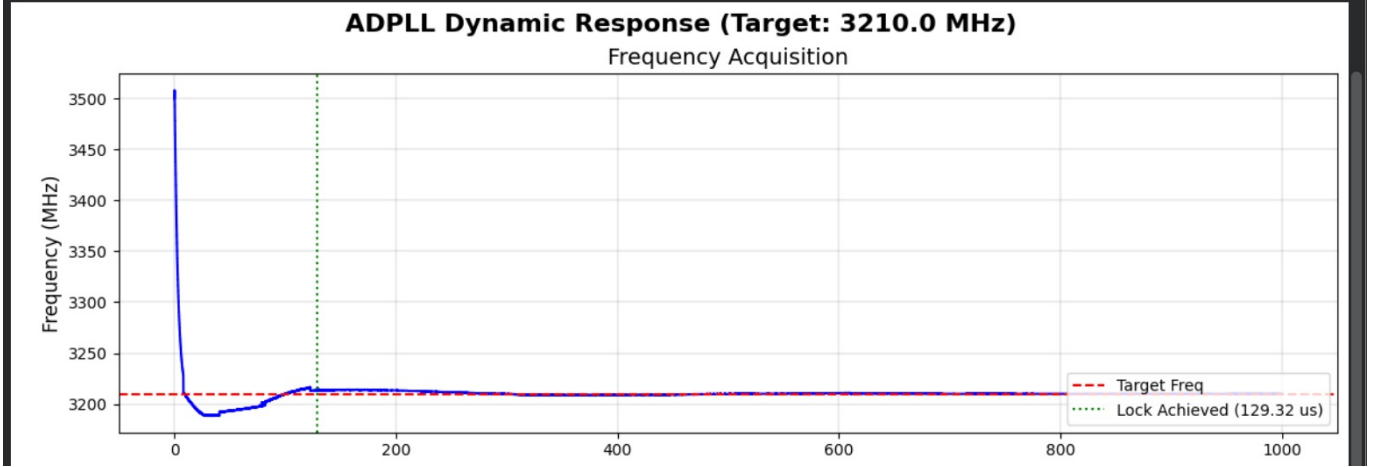


Figure 2: High Frequency ADPLL output

4 FPGA Implementation Challenges and Constraints

Firstly, the target output frequency of 3-4GHz fundamentally exceeds the maximum operating speed of FPGAs because their clock distribution networks have a maximum toggle rate between 600-800MHz. Moreover, the proposed ADPLL architecture requires a DCO (Ring Oscillator) operating in the multi-gigahertz frequency range. However, standard synthesis tools typically interpret the asynchronous combinational loops required for a ring oscillator as timing violations or design errors. Even if synthesized, FPGA implementations are restricted to routing and Look-Up Table (LUT) propagation delays, which yield coarse and highly non-linear frequency steps. As a result, direct implementation of the high-frequency ADPLL model was not feasible, since it would lead to unreliable operation.

5 FPGA-Oriented ADPLL Architecture

To enable FPGA implementation the entire ADPLL model was scaled down by a factor of 50. The reference frequency, target output frequency, DCO free-running frequency, DCO gain (K_{DCO}) and all other constants in the frequency domain were scaled down where as TDC resolution, DTC resolution and all other parameters in the time domain were scaled up by the same factor. This was done to keep the loop dynamics unchanged. We then performed the implementation, gate level simulation, jitter analysis, etc with the expectation that the results would be representative of the original design.

Moreover, Ring oscillator can't be implemented accurately on an FPGA it was replaced with a Numerically controlled oscillator(NCO). A NCO generates a stable, variable-frequency signal by continuously adding a specific frequency tuning word (FTW) to a phase accumulator at every cycle of a fixed, high-speed physical clock. This continuous addition creates a digital phase value that accumulates up over time. The final output clock or waveform is produced by extracting the most significant bits of this accumulated phase. However to sample this phase accumulator we require a high frequency clock which is 400MHz (period of 2.5ns) in our model. Thus our output clock period is quantized to multiples of 2.5ns (2.5, 5, 7.5...).

6 Bonus Tasks

6.1 Clock Domain Crossing (CDC) via Asynchronous FIFO

In modern SoCs, transferring data between a slow external reference clock and a high-speed generated clock risks metastability and data corruption. To safely bridge this clock domain crossing, an Asynchronous First-In, First-Out (FIFO) buffer was implemented using a dual-port RAM architecture. The write logic is strictly driven by the slow reference clock, while the read logic is driven by the fast DCO clock. To prevent multi-bit transition errors when evaluating the FIFO status across these asynchronous domains, the binary read and write memory pointers are converted into Gray code. These Gray-coded pointers are then passed through standard two-stage flip-flop (2-flop) synchronizers before entering the opposing clock domain to safely compute the empty and full flags.

To verify the robustness of the CDC architecture, a testbench was designed to continuously write a sequence of incrementing data packets into the FIFO at the slow reference frequency, while simultaneously reading them out using the newly locked, fast DCO clock. The simulation waveform confirms that the empty and full flags assert correctly to regulate data flow. Most importantly, the incrementing data sequence exits the fast domain perfectly intact without any dropped, duplicated, or metastable bits, successfully validating the domain isolation.

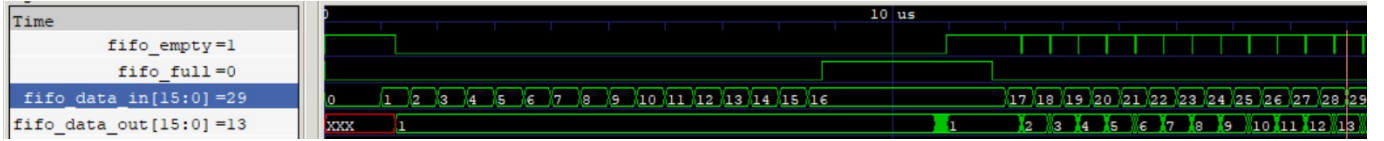


Figure 3: Simulation waveform showing empty and full flags and proving that data sequence exits perfectly without any lost, dropped, repeated bits.

6.2 Adaptive "Gear Shifting" Loop Filter

To overcome the fundamental trade-off between rapid phase acquisition (which requires a wide loop bandwidth) and low steady-state jitter (which requires a narrow bandwidth), an adaptive "Gear Shifting" loop filter was implemented. The standard Proportional-Integral (PI) digital filter was modified to use dynamic multiplexers that adjust arithmetic right-shift amounts, effectively changing the K_p and K_i coefficients without utilizing resource-heavy DSP multipliers. A dynamic gain scheduler with a three-tier shifting mechanism was added: Fast Acquisition (Large bandwidth), Mid-Range Tracking (Medium bandwidth), and Fine-Tuning (Narrow bandwidth). The active mode is determined continuously based on the magnitude of the phase error. To ensure structural stability, hysteresis was also added into the state machine's thresholding logic, which prevents the system from jittering between two modes when the phase error fluctuates near a boundary. The transient simulation waveform validates this dynamic gear shifting, clearly displaying the phase error shrinking rapidly and the change in K_p , K_i values based on phase error.

Table 1: Adaptive Gear Shifting Loop Filter Parameters

Operating Mode	Phase Error Threshold (<)	Loop Bandwidth ($\approx$)	Damping Factor ($\zeta \approx$)
Fast Acquisition	—	12 kHz	0.7
Medium Range	2000	6 kHz	0.8
Fine Tracking	1000	3 kHz	0.9

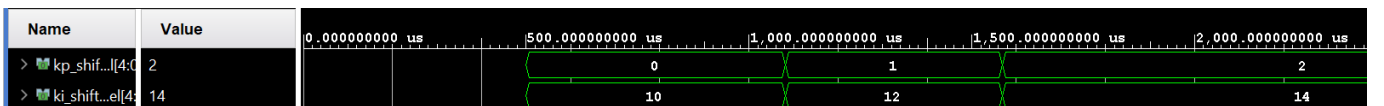


Figure 4: Closed-loop transient simulation demonstrating gear shifting

6.3 Fractional-N Synthesis via Sigma Delta Modulation

An Integer-N PLL can generate output frequencies only at integer multiples of reference frequency. To achieve fractional multiplication ratios, a MASH 1-1 Delta-Sigma Modulator was incorporated into the clock divider path. The modulator

dynamically changes the divider ratio while maintaining the required average division ratio. Quantization noise introduced by fractional division is shaped and pushed to higher frequencies, reducing in-band phase noise.

The frequency plot below uses FFT analysis to evaluate the performance of the MASH 1-1 $\Delta\Sigma$ modulator. When a PLL uses fractional division, it naturally produces small quantization errors. By observing the data in the frequency domain through the use of FFT, we can see how the digital design addresses this issue. The x axis shows the frequency offset from the target frequency and y axis the corresponding frequency noise density. The drop in the noise floor between 10 kHz and 40 kHz indicates that the noise closest to our carrier frequency is significantly reduced, helping to keep the main signal band clear.

Since quantization error cannot be entirely eliminated, the modulator relies on noise shaping to redistribute it to higher frequencies. This effect is visible on the right side of the graph, where the noise floor slopes upward from 100 kHz toward the 1 MHz boundary. This rising slope demonstrates that much of the remaining error is being shifted into higher frequencies. Overall, these results suggest that the 2nd-order digital design is functioning as intended, contributing to a more stable closed-loop signal.

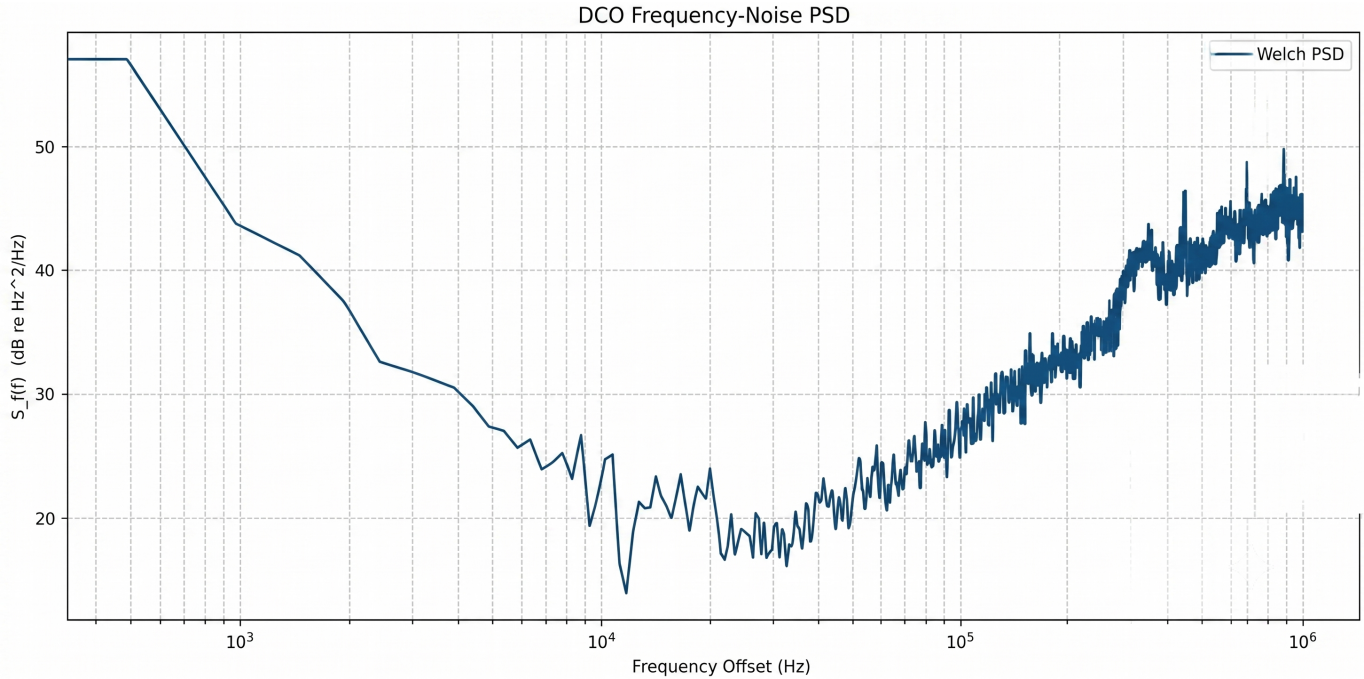


Figure 5: Fast Fourier Transform Analysis

7 Extra features

7.1 Frequency Locked Loop(FLL)

The FLL mechanism starts of with an edge counter that counts the rising edge of the DCO clock, measuring the absolute count of the DCO clock. This needs to be converted to the reference clock domain. This is achieved by Gray-coding the DCO count and passing it through a 2-stage flip flop synchronizer. Once in reference clock domain, the Gray-code is converted back to binary. The system uses this value to perform a measurement over a 16-cycle reference window. It subtracts the saved value from the previous 16-cycle window to get the exact count for the current 16-cycle window. Since the values are accumulated over 16 reference cycles, an expected target value ($target_{16}$) is calculated by scaling the input N_{int} using a shift operator and computing the fractional K_{mod} contribution using fixed-point multiplication and shifting. The DCO count and target value are subtracted to get FLL frequency error. This error is then used to calculate the FLL control word. This loop continues until the frequency error remains within a specific threshold for consecutive windows, after which the FLL hands over the control to the PLL.

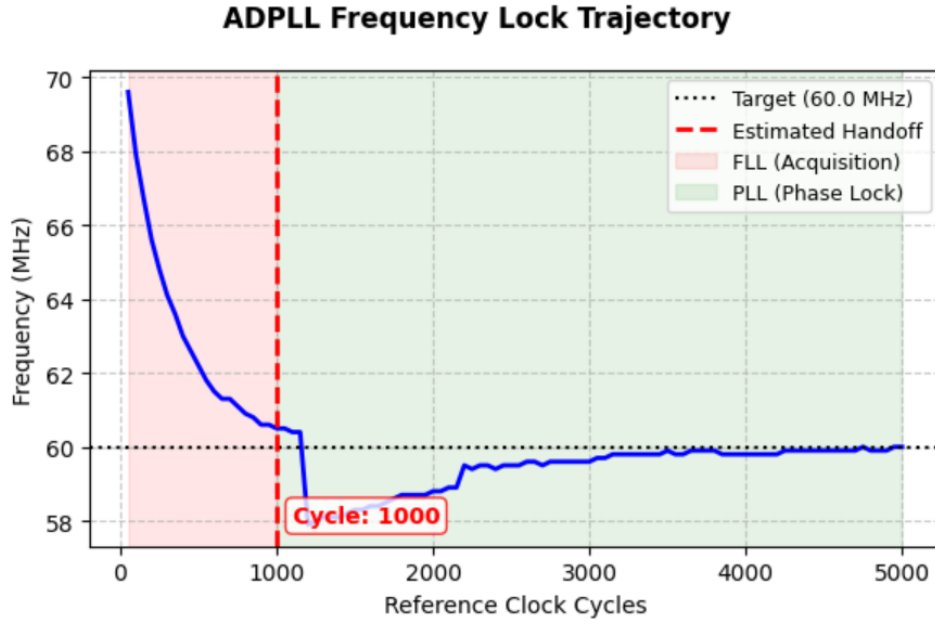


Figure 6: FLL active period

7.2 CIC Decimator

The CIC (Cascaded Integrator-Comb) Decimator serves three primary functions in this architecture: it acts as a downsampler, a low-pass filter, and a lock/power manager. The phase error arrives at the reference clock rate from the phase detector and TDC, which would cause unnecessary dynamic power consumption if processed continuously by downstream blocks. To optimize energy efficiency, the CIC decimator downsamples this data rate by a factor of 4 ($R = 4$). It also functions as a low-pass filter to attenuate high-frequency quantization noise and jitter originating from the TDC. It achieves this by using two cascaded 29-bit accumulators that continuously sum the 25-bit phase-residual input at full clock speed. This creates a mathematical moving-average low-pass filter with a sinc-like frequency response:

$$H(z) = \left(\frac{1 - z^{-R}}{1 - z^{-1}} \right)^2$$

The comb stage then uses differentiators at the decimated rate to compute the differences between consecutive integrated samples.

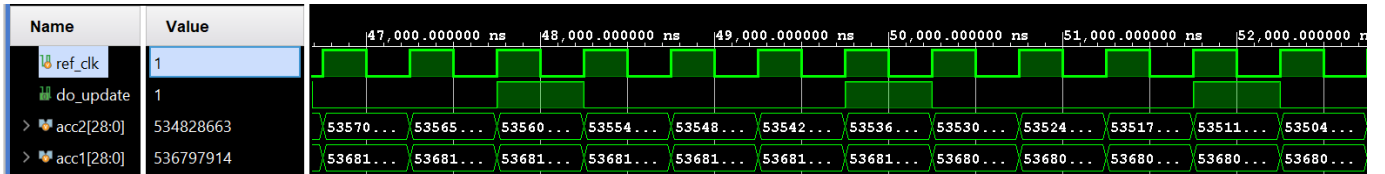


Figure 7: Both Accumulators from CIC

7.3 Power Optimization

Integer-Mode Power Optimization: To reduce power consumption, the architecture disables/switches off the Mash modulator and the DTC when the ADPLL is in Integer N mode ($K_{\text{mod}} = 0$).

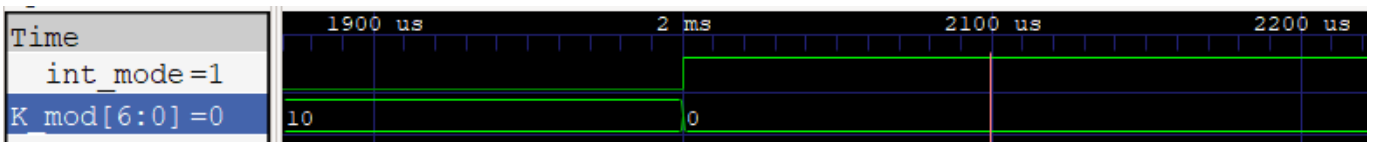


Figure 8: Simulation waveform shows the int_mode flag going high when K_mod is changed to 0.

Low-Power Mode: To further optimize power efficiency during steady-state operation, a low-power mode was implemented. The system actively monitors the phase error; when the error remains consistently below a threshold of 1000 for 3000 consecutive cycles, the lock detector triggers low-power mode. In this state, the phase detector and accumulators remain operational to continuously track phase drift, but computationally heavy PI filter is run once in 8 cycles. This reduction in filter switching activity reduces dynamic power while maintaining system synchronization. If external noise or disturbance causes the phase error to exceed the threshold, the system instantaneously exits low-power mode.

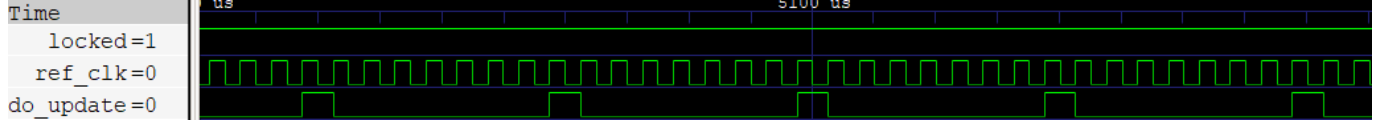


Figure 9: Simulation waveform shows system in low power mode, where the PI filter is updated once in 8 cycles.

8 Results and Performance Metrics

8.1 System Operating Parameters

Table 2: ADPLL Operating Parameters

Parameter	Value
Reference Clock Frequency (F_{ref})	2 MHz
DCO Free-Running Frequency	70 MHz
Output Frequency Range	60 MHz – 80 MHz
Frequency Resolution	0.4 kHz
Loop Bandwidth	3kHz - 12kHz
Damping Factor	0.7 - 0.9

8.2 Transient Response and Lock Dynamics

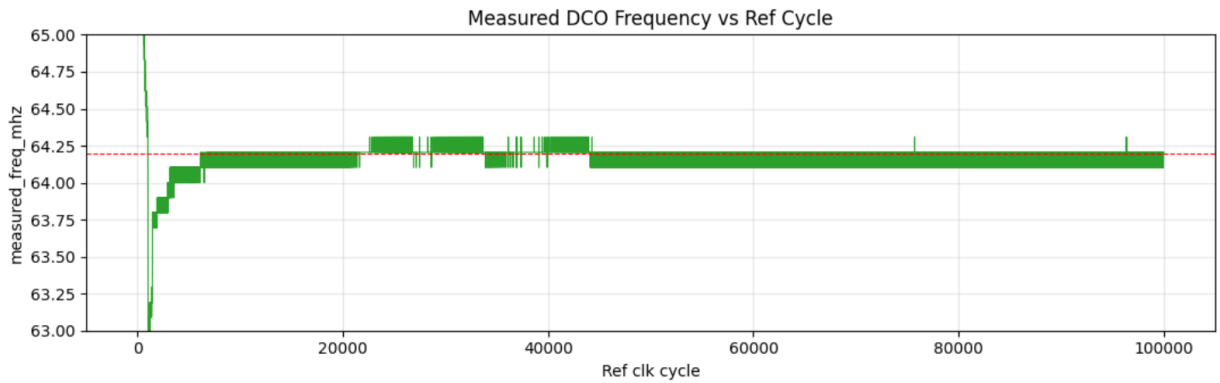


Figure 10: Graph showing the DCO output frequency smoothly converge at the target(64.2MHz)

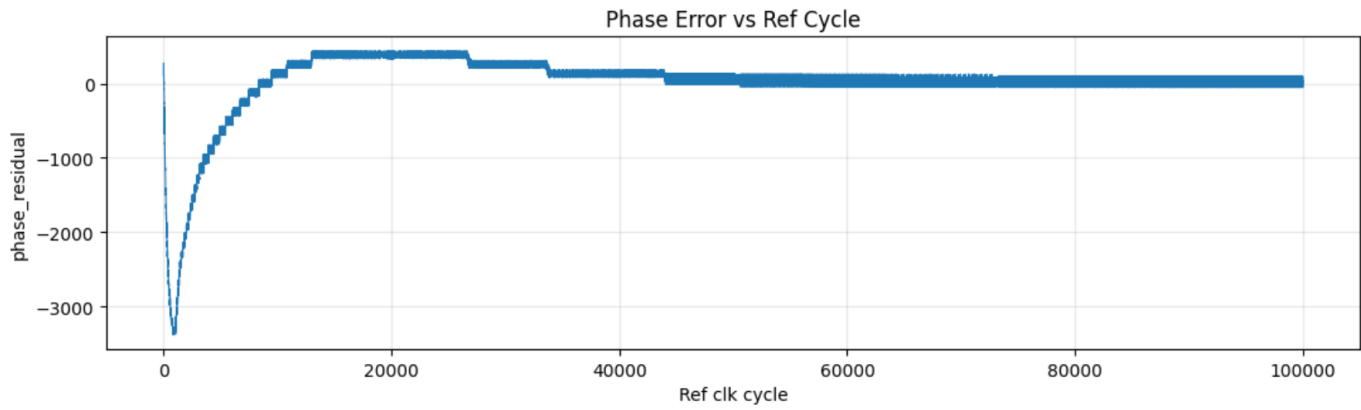


Figure 11: Graph showing the phase error smoothly converge to 0

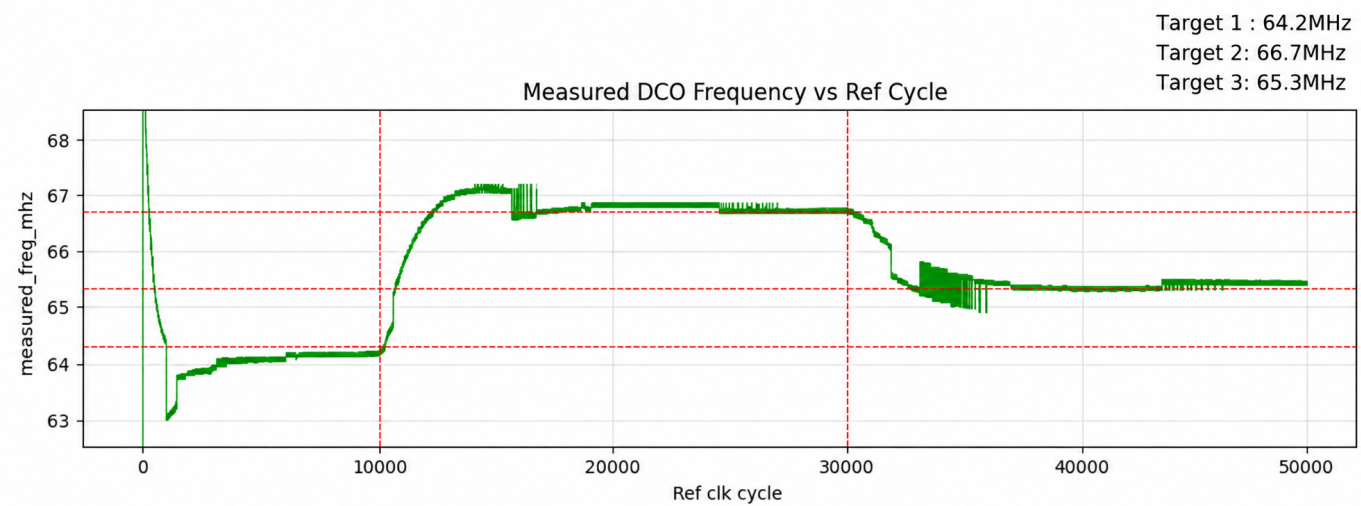


Figure 12: Graph showing the step response of the ADPLL.

Table 3: Transient Lock Performance and Frequency Accuracy

Metric	Value
Lock Time (Cold Start / Reset)	~ 2500 reference cycles
Lock Time (Target Frequency Step)	~ 7000 reference cycles
Maximum Steady State Frequency Error	0.2 MHz
Limit cycle Oscillations Amplitude	0.1 MHz

Upon system reset, the ADPLL utilizes the frequency locked loop(FLL) alongside wide-bandwidth to rapidly acquire coarse frequency alignment, reaching a locked state within approximately 2500 reference cycles. When subjected to an operational target frequency change, the system uses the PLL only. It smoothly adjusts through intermediate gain-scheduled states, achieving final phase lock within approximately 7000 cycles.

8.3 Vivado Implementation

The power report generated here is a **High** confidence power report generated after recording the Switching Activity Interchange Format (SAIF). It covers 83% of the netlist. The recorded dynamic power is 15 mW, which is well below

the expected range. The remaining 70mW is coming from the leakage current from the unused LUTs. This number will further go down with clock gating and proper routing if implemented on an ASIC design.

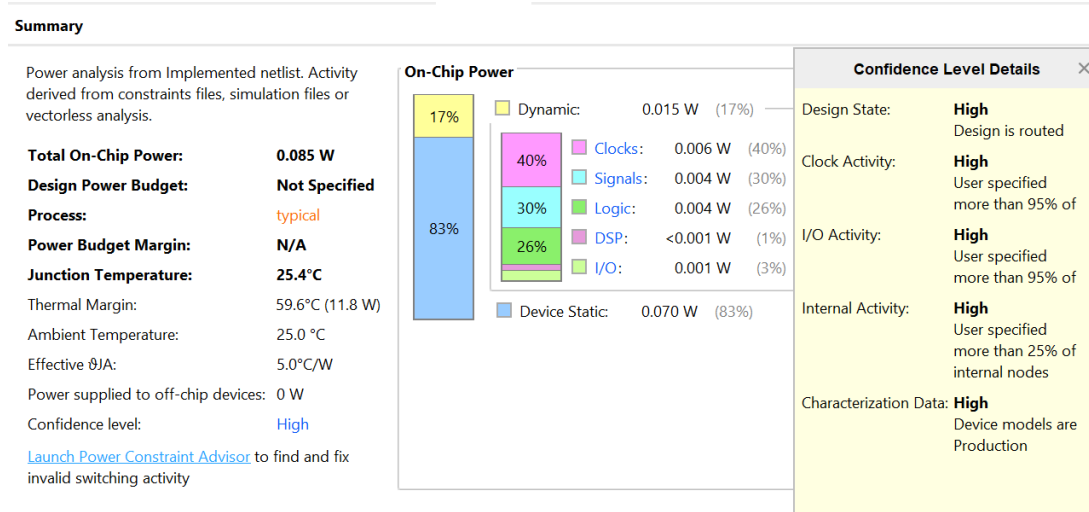


Figure 13: Vivado Power Report

Moving on to the utilization report, the system is highly optimized in terms of bit width and LUT usage. The bit widths used are tightly kept to avoid excess bits driven. However silicon use increases in certain modules like DCO and PI-Filter which require pipelining to keep the logic up with clock speed, this is reflected in the utilization report

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	DSPs (90)	Bonded IOB (106)	BUFGCTRL (32)
adpll_top	1019	731	345	1019	4	65	3
cic_inst (cic_decimator)	301	153	107	301	0	0	0
fast_fll_inst (direct_fll)	113	85	42	113	1	0	0
filter_inst (pi_loop_filter)	229	123	85	229	0	0	0
inst (dco_nco)	100	144	41	100	1	0	0
mash_inst (mash_modulator)	68	21	22	68	0	0	0
pd_inst (phase_detector)	103	108	54	103	0	0	0
scheduler_inst (gain_scheduler)	64	32	40	64	0	0	0

Figure 14: Vivado Utilization Report

Timing analysis reveals the result of the pipelining done. The logic is comfortably able to pass through with some room to spare. The slowest path(critical path delay) are between two pipelining registers in the DCO.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.188 ns	Worst Hold Slack (WHS): 0.119 ns	Worst Pulse Width Slack (WPWS): 0.345 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1009	Total Number of Endpoints: 979	Total Number of Endpoints: 735
All user specified timing constraints are met.		

Figure 15: Vivado Timing Report

8.4 Jitter and Time-Domain Analysis

Table 4: Time-Domain Output Clock Stability and Jitter Metrics

Metric	Value
Average Period (T_{avg})	311.493 ps
Minimum Period (T_{min})	311.126 ps
Maximum Period (T_{max})	311.684 ps
Peak-to-Peak Period Jitter ($\Delta T_{\text{p-p}}$)	0.558 ps
RMS Period Jitter (σ_J)	0.125 ps

The time-domain characteristics of the DCO clock demonstrate high edge stability across continuous operation. The average clock period corresponds to the target output frequency, while the minimum and maximum period deviations define the worst-case peak-to-peak jitter boundary. The low RMS period jitter confirms that phase noise and quantization effects introduced by the fractional control logic remain tightly bounded during steady-state tracking.

It is important to note that these reported jitter metrics are extracted from the high-resolution behavioral model rather than the post-synthesis FPGA implementation. In the FPGA oriented module, the ring oscillator is replaced with a Numerically Controlled Oscillator (NCO) running off a 400 MHz board clock ($T_{\text{clk}} = 2.5$ ns). Because the output period in the FPGA implementation is quantized in integer multiples of 2.5 ns, any direct jitter measurement on the FPGA model would be heavily dominated by this clock-quantization artifact, masking the true noise performance of the ADPLL loop.

8.5 Gate Level Simulation

Gate-Level Simulation was performed so that the testbench exercised the design against actual silicon/FPGA propagation delays rather than the zero-delay assumption of RTL simulation. This was necessary because RTL simulation only verifies logical correctness and cannot account for real gate and interconnect delay — factors that directly affect loop timing and, output jitter in a closed-loop system like the ADPLL. GLS thus served as the final verification step to confirm the design’s functional and timing behavior under realistic delay conditions.

GLS confirms functional correctness and timing closure of the ADPLL under real gate/wire delays. Due to the NCO’s 2.5 ns output quantization, sub-quantization jitter effects from implementation delay are not observable at the output. Instead, loop settling time and control-word ripple were used as delay-sensitive proxies, and no significant degradation was observed in these.

9 Limitations and Future Work

There are many improvements that could have been made but were avoided due to a lack of resources and time. For instance, the FLL can be expanded further to activate every time there is a change in the required frequency. As seen in the step response graph (Fig. 12), when the target frequency is changed from 64.2 MHz to 66.7 MHz, and subsequently to 65.3 MHz, it takes approximately 7000 cycles to reach the new target. This settling time could be significantly reduced if the FLL were to activate during each of these frequency transitions.

The model is currently constrained by an FPGA for which many changes had to be made. Silicon usage had to be increased tremendously to pipeline the logic which is slow on an FPGA. DCO had to be replaced with a NCO which constrained by the board clock, again slow on an FPGA. Overcoming this will involve shifting to a ASIC implementation where one is not constrained to use predefined paths as on an FPGA. The DCO can be fully implemented as a ring oscillator which will give the exact output rather than the current output is constrained by the precision offered by the board clock (2.5ns). ASIC will allow the design to be run at much higher speeds rather than the scaled version here (scaled down by 50x). Further deep clock gating can be implemented in ASIC as we have control over the routing of the clock tree.

Improvements can be made on certain modules like the PI-Filter and mash-modulator. Currently the second order PI-Filter in the code can provide up to -20 DB/decade. increasing the order will lead to a faster roll off and reduce capture time. Currently mash 1-1 modulator is used, which has two 1st order accumulators. This does a decent job in

reduction of noise but a mash 1-1-1 modulator has a cubic transfer function and can reduced quantitation noise more aggressively.

Currently the K_{DCO} is constant however in real chips K_{DCO} fluctuates significantly with Process, Voltage, and Temperature (PVT) variations. Implementing a dynamic K_{DCO} is needed to overcome these challenges. This will involve adding an LMS (Least Mean Squares) background calibration loop which solves this dynamically.

10 Conclusion

In this project, a high-performance, low-power All-Digital Phase-Locked Loop (ADPLL) architecture was successfully designed and verified in Verilog. Synthesis and implementation were done in the AMD Xilinx Vivado Suite. The design demonstrated excellent performance and energy efficiency. It achieves a total dynamic power consumption of just 15 mW under SAIF-annotated simulation. This confirms the design's viability for modern high-speed clock generation and low-power SoC integrated circuits. The project was bound to an FPGA, which limited its capabilities. Further work can be done to make this project ASIC-oriented, allowing for a faster and more accurate version.